API reference

The request/response detail is **not hand-written here** — it is generated from the route definitions, so it cannot drift from the code:

What	Where
Interactive docs (Swagger UI)	`http://localhost:8000/docs` while the backend runs
ReDoc	`http://localhost:8000/redoc`
Machine-readable schema	`http://localhost:8000/openapi.json`
Snapshot committed to the repo	`openapi.json` (OpenAPI 3.1, 42 paths / 52 operations / 70 schemas)

Every operation carries a `summary`, a `description` (rules, side effects, role requirements) and its error responses; every request field has a description. Import `openapi.json` into Postman, Insomnia or a client generator if you prefer.

Regenerating the snapshot

The snapshot is produced from the app object itself, no server needed:

```
cd backend
python -c "import json; from app.main import app; json.dump(app.openapi(),
open('../docs/api/openapi.json','w',encoding='utf-8'), indent=2, ensure_ascii=False)"
```

Regenerate it whenever a route or schema changes. A quick consistency check is to compare it with the live `/openapi.json`.

Trying it in Swagger UI

1. `POST /auth/login` with `{"email": "admin@example.com", "password": "ChangeMe123!"}` (the seeded **platform admin**, see local setup) and copy `access_token`. To try company endpoints, log in as a company user created by the platform admin instead.
2. Click **Authorize** (top right), paste the token (Swagger adds `Bearer` for you).
3. Every other call now sends it.

Conventions

- **Auth:** `Authorization: Bearer <JWT>`. Access tokens live 15 minutes by default (`JWT_ACCESS_TOKEN_EXPIRE_MINUTES`); the session is renewed with `POST /auth/refresh` (refresh token in an httpOnly cookie — see authentication). Tokens carry `role`, `company_id`, `is_platform_admin` and `ver`. The user is re-read from the database on every request; a token whose company or platform flag no longer matches, or whose company is suspended, is refused. A missing/invalid/expired token or an inactive user is **401**; a valid user with the wrong role is **403**.
- **Two mandatory checks on every endpoint:** the role check and the tenant check. A company user's request is confined to their own company — by the application and by PostgreSQL Row-Level Security.
- **Company roles (ranked):** `user` (submitter) < `reviewer_l1` < `reviewer_l2`. Each role can do everything the roles below it can; routes name the lowest role they admit. On a case escalated to L2 (`assigned_tier` = `l2`) a `reviewer_l1` can still `GET` it but gets **403** from approve/reject/escalate.

- `platform_admin` belongs to no company and is not on that ladder. It manages companies, users, the rule template, billing and the queue monitor, may soft-delete and restore any company's case (`DELETE /cases/{id}` , `POST /cases/{id}/restore`), and has **read-only, audited** access to any company's data by passing `company_id` (query parameter) on the company endpoints that accept it — `GET /cases` , `/audit-log` , `/settings/*` — or simply opening a case by id. It can never approve, reject, escalate, upload, create cases or generate reports (403), but may edit a company's issuer registry, risk rules and thresholds on its behalf. Everything a platform admin does is audited **platform-only** (`company_id` NULL): no company user ever sees an admin's access or changes.
- **A resource of another company is a 404**, and so is a case that is not yours for a `user` — ids cannot be probed.
- **Ids** are UUIDs. **Timestamps** are ISO-8601 with timezone.
- **Bounding boxes** are always `{page (1-based), x, y, width, height}` as 0–1 fractions of the page, origin top-left.
- **Errors** are `{"detail": "<human-readable message>"}` (or FastAPI's standard list for **422** validation errors). **Upload rejections** are structured: `{"detail": {"code": …, "message": …, …}}` (see 01-upload-intake). Status codes used: 400, 401, 403, 404, 409 (state conflict or duplicate), 413 (upload too large), 415 (not a PDF / wrong extension), 422 (validation, business rule, corrupted or password-protected PDF), 502 (Azure Blob Storage failure).
- **There is no `/api/v1` prefix in the backend.** `API_V1_PREFIX` is declared in settings but never used. In development the Vite dev server proxies `/api/*` → `http://127.0.0.1:8000/*` , so the browser calls `/api/cases` while the backend sees `/cases` . Production serves the SPA and the API behind one ingress the same way (production deployment).
- **Pagination:** only `GET /audit-log` is paged (`limit` 1–500, default 100, and `offset` ; response is `{total, items}`). Other lists return everything.
- **No dedicated "checks" endpoints.** Per-document check results, extracted fields, cross-document findings, forensic findings, the risk assessment and reviewer actions are all embedded in `GET /cases/{case_id}` . Signature comparison results have their own endpoint.
- **Asynchronous work:** uploads and signature-reference creation return before processing finishes. Poll `GET /cases/{case_id}` (`pipeline.complete` , `documents[].processing_status`) and `GET /cases/{case_id}/signature-matches` .

Endpoint index

`Access` shows who may call each endpoint. "own case" means a `user` may act only on a case they submitted (anyone else gets a 404), while reviewers (L1 or L2) may act on any case of their company. "reviewer" means `reviewer_l1` or above. "PA read" means a platform admin may read it (audited). See the security notes in architecture/overview.md.

Auth — tag `auth`

Method & path	Summary	Access
`POST /auth/login`	Log in and obtain a JWT	public
`GET /auth/me`	Get the current user (with company)	any
`GET /auth/me/upload-limits`	The caller's company upload limits (max file / zip size), read fresh each time	company users (404 for platform admins)

Cases — tag `cases`

Method & path	Summary	Access
`POST /cases`	Create a case	company users

Method & path	Summary	Access
`GET /cases`	List cases (the case queue); filters `status`, `case_type`, `assigned_tier`, `actionable`; each item has `can_act`	company users (`user` sees only own); PA read with `company_id`
`GET /cases/page`	One page of cases (`limit` ≤ 200, `offset`) with `total`; filters `status_group`, `risk`, `case_type`, `assigned_tier`, `actionable`, `mine`, search `q` (case number, submitter name/email), `sort=priority`. Used by the case queue, dashboard and My Cases	same as `GET /cases`
`GET /cases/summary`	Counts for the queue, dashboard and My Cases cards (status groups, risk tiers, documents, tab counts); `mine=true` for own submissions. A `user` gets no risk breakdown	same as `GET /cases`
`GET /cases/{case_id}`	Get case detail (documents, checks, findings, assessment, actions)	company users (`user` sees a reduced view); PA read
`GET /cases/{case_id}/audit-log`	Get a case's activity timeline	own case for `user`, reviewers; PA read

Reviewer decisions — tag `case-actions`

Method & path	Summary	Access
`POST /cases/{case_id}/approve`	Approve a case (justification required above low risk)	reviewer; L2 case: `reviewer_l2`
`POST /cases/{case_id}/reject`	Reject a case (reason required)	reviewer; L2 case: `reviewer_l2`
`POST /cases/{case_id}/escalate`	Escalate a case to the L2 tier (reason required; one-way; 409 if already L2)	reviewer on an L1 case

Documents — tag `documents`

Method & path	Summary	Access
`POST /cases/{case_id}/documents`	Upload a document (multipart field `file`, one per request; **PDF only**, within the company's own size limit (10 MB by default; set per company by a platform admin), not password-protected; rejections return `detail: {code, message}`, see 01-upload-intake)	own case, or reviewer
`POST /bulk-uploads`	Bulk upload: one zip (raw body, `Content-Type: application/zip`; query `case_type`, `filename`), one folder per case, within the company's zip limit. 202 with the per-case plan; ingestion runs in the background (01a)	any company role
`GET /bulk-uploads`	Recent bulk uploads	own (user) / company (reviewers) / platform admin per company

Method & path	Summary	Access
`GET /bulk-uploads/{id}`	Bulk upload summary with each case's live status	same as above
`GET /cases/{case_id}/documents/{document_id}/file-url`	Get a fresh signed URL for the original file	own case, or reviewer; PA read

Signatures — tag `signatures`

Method & path	Summary	Access
`POST /cases/{case_id}/documents/{document_id}/signature-references`	Create a signature reference from a drawn box	own case, or reviewer
`GET /cases/{case_id}/signature-references`	List a case's references	own case, or reviewer; PA read
`GET /cases/{case_id}/signature-matches`	List a case's comparison results	own case, or reviewer; PA read

Reports — tag `case-reports`

Method & path	Summary	Access
`POST /cases/{case_id}/reports`	Generate a case report (PDF)	reviewer
`GET /cases/{case_id}/reports`	List a case's generated reports	reviewer; PA read

> Organisation-wide MIS/compliance reporting (`app/api/reports.py`) is a stub with no routes.

Audit — tag `audit`

Method & path	Summary	Access
`GET /audit-log`	Search the company's audit log (`event_type` , `case_id` , `q` , `limit` , `offset`); never shows platform-access rows to a company user	reviewer; PA with `company_id` (that company's log incl. platform-access rows about it) or without (the platform-level log, which also lists every company's settings changes; each item carries `company_name` for PAs)
`GET /audit-log/event-types`	List distinct audit event types	reviewer; PA

Settings — tag `settings`

Issuer registry, risk rules and thresholds are **per company**: a company's `reviewer_l2` , or a platform admin passing `company_id` (audited). Users are platform-admin only.

Method & path	Summary	Access
`GET /settings/issuers`	List issuers (`include_inactive`)	`reviewer_l2` ; PA

Method & path	Summary	Access
`POST /settings/issuers`	Create an issuer	`reviewer_l2` ; PA
`PATCH /settings/issuers/{issuer_id}`	Update or deactivate an issuer	`reviewer_l2` ; PA
`GET /settings/risk-rules`	List current risk rules	`reviewer_l2` ; PA
`POST /settings/risk-rules`	Create a risk rule	`reviewer_l2` ; PA
`GET /settings/risk-rule-options`	Options for the Add-rule form	`reviewer_l2` ; PA
`GET /settings/risk-rules/{rule_id}/history`	A rule's version history	`reviewer_l2` ; PA
`PATCH /settings/risk-rules/{rule_id}`	Change weight / severity / active flag (inserts a new version)	`reviewer_l2` ; PA
`GET /settings/risk-thresholds`	Get the tier thresholds	`reviewer_l2` ; PA
`PUT /settings/risk-thresholds`	Set the tier thresholds	`reviewer_l2` ; PA
`GET /settings/users`	List users (filter `company_id`)	PA only
`POST /settings/users`	Create a user (company + role, or a platform admin)	PA only
`PATCH /settings/users/{user_id}`	Change role / company / active flag (moving company invalidates the user's token)	PA only

Platform — tag `platform` (platform admins only)

Method & path	Summary
`GET /platform/companies`	List companies
`POST /platform/companies`	Create a company (seeds its risk rules from the template and default thresholds)
`PATCH /platform/companies/{company_id}`	Rename / suspend / reactivate a company, or change its upload limits (`max_file_size_mb` , `max_zip_size_mb` , `max_file_pages` ; audited old → new)
`GET /platform/usage`	Billing & usage per company (`period` or a custom date range)
`POST /platform/usage/reconcile`	Recompute the usage counters now
`GET /platform/queues`	Processing-queue monitor (depth, waits, workers, limiter counters, outstanding per company)
`GET /platform/risk-rule-templates`	List the platform rule template
`POST /platform/risk-rule-templates`	Add a template rule (affects companies created afterwards)

Method & path	Summary
`PATCH /platform/risk-rule-templates/{rule_id}`	Change a template rule (never changes existing companies)

Health — tag `health`

Method & path	Summary	Access
`GET /health`	Liveness check (does not touch DB/Redis/Azure)	public

Not in the API (by design or not yet built)

No password reset or change-password, no self-service sign-up, no logout/refresh, no company-level user management (users are created by platform admins), no document delete (originals are immutable), no case delete, no notification endpoints, no organisation-wide reporting, no external-integration hooks.